

Training Python-Django - 2

ORM

Summary: Today, we'll tackle the ORM of `Django`.

Version: 1.1

Contents

I	General rules	2
II	Today's Specific Rules	3
III	Exercise 00	5
IV	Exercise 01	6
V	Exercise 02	7
VI	Exercise 03	9
VII	Exercise 04	11
VIII	Exercise 05	13
IX	Exercise 06	15
X	Exercise 07	17
XI	Exercise 08	19
XII	Exercise 09	21
XIII	Exercise 10	24
XIV	Submission and peer-evaluation	26

Chapter I

General rules

- Your project must be realized in a virtual machine.
- Your virtual machine must have all the necessary software to complete your project. These softwares must be configured and installed.
- You can choose the operating system to use for your virtual machine.
- You must be able to use your virtual machine from a cluster computer.
- You must use a shared folder between your virtual machine and your host machine.
- During your evaluations you will use this folder to share with your repository.
- Your functions should not quit unexpectedly (segmentation fault, bus error, double free, etc) apart from undefined behaviors. If this happens, your project will be considered non functional and will receive a `0` during the evaluation.
- We encourage you to create test programs for your project even though this work **won't have to be submitted and won't be graded**. It will give you a chance to easily test your work and your peers' work. You will find those tests especially useful during your defence. Indeed, during defence, you are free to use your tests and/or the tests of the peer you are evaluating.
- Submit your work to your assigned git repository. Only the work in the git repository will be graded. If Deepthought is assigned to grade your work, it will be done after your peer-evaluations. If an error happens in any section of your work during Deepthought's grading, the evaluation will stop.

Chapter II

Today's Specific Rules

- You must use a `python3` interpreter.
- Each exercise will be independent. If some required features have already been implemented in previous exercises, you must duplicate them in the current one.
- You must work with a `PostgreSQL` database named `djangotraining` and create a role named `djangouser`, whose password is `"secret"`, and which will have all the rights on it.
- Your repository folder must be a Django project. The project must be named after the current day.
- We will use Django's application concept to separate the exercises:
  Today's exercises must be located in a specific Django application named after the corresponding exercise and located at the root of the repository.
- The Django project must be properly configured to meet the exercise's requirements. You will not be able to change the configuration during evaluation.
- You must not submit any migration files with your work.
- In each exercise whose card mentions `ORM`, you must use Django's ORM. You must not write any line of `SQL`.
- In each exercise whose card mentions `SQL`, you must use the `psycopg2` library and run all requests in `SQL`.

Here is a typical example of a repository structure for a student named krichard, for day d42, including 2 exercises:

```
|-- krichard
|   |-- .
|   |-- ..
|   |-- .git
|   |-- .gitignore
|   |-- d42
|   |   |-- __init__.py
|   |   |-- settings.py
|   |   |-- urls.py
|   |   |-- wsgi.py
|   |-- ex00
|   |   |-- admin.py
|   |   |-- apps.py
|   |   |-- forms.py
|   |   |-- __init__.py
|   |   |-- models.py
|   |   |-- tests.py
|   |   |-- urls.py
|   |   |-- views.py
|   |-- ex01
|   |   |-- admin.py
|   |   |-- apps.py
|   |   |-- forms.py
|   |   |-- __init__.py
|   |   |-- models.py
|   |   |-- tests.py
|   |   |-- urls.py
|   |   |-- views.py
|   |-- manage.py
```



Be smart: factor your code and make it easy to use. You will save some time.

Chapter III

Exercise 00

	Exercise 00
Exercise 00: SQL - Building a Table	
Turn-in directory : *ex*00/	
Files to turn in :	
Allowed functions :	

Create a Django application named `ex00` as well as a view within it that must be accessible at the following URL: `127.0.0.1:8000/ex00/init`.

This view must create an SQL table in PostgreSQL using the `psycopg2` library and return a page containing "OK" if successful. Otherwise, it must return an error message describing the problem.

The SQL table must meet the following specifications:

- It must be named `ex00_movies`.
- It must be created only if it does not already exist.
- It must contain only the following fields:
  - `title`: unique, variable-length character string, maximum size of 64 bytes, not null.
  - `episode_nb`: integer, PRIMARY KEY.
  - `opening_crawl`: text, may be null, with no size limit.
  - `director`: variable-length character string, not null, maximum size of 32 bytes.
  - `producer`: variable-length character string, not null, maximum size of 128 bytes.
  - `release_date`: date (without time), not null.

Chapter IV

Exercise 01

	Exercise 01
Exercise 01: ORM - Building a Table	
Turn-in directory : *ex*01/	
Files to turn in :	
Allowed functions :	

Create an application named `ex01`. In it, create a `Django` model named `Movies` with exactly the following fields:

- `title`: unique, variable-length character string, maximum size of 64 bytes, not null.
- `episode_nb`: integer, PRIMARY KEY.
- `opening_crawl`: text, may be null, with no size limit.
- `director`: variable-length character string, not null, maximum size of 32 bytes.
- `producer`: variable-length character string, not null, maximum size of 128 bytes.
- `release_date`: date (without time), not null.

This model must also redefine the `__str__` method so that it returns the value of the `title` attribute.

Chapter V

Exercise 02

	Exercise 02
Exercise 02: SQL - Data Insertion	
Turn-in directory : *ex*02/	
Files to turn in :	
Allowed functions :	

You must create a Django application named `ex02`. In this application, the following views must be accessible via the following URLs:

- `127.0.0.1:8000/ex02/init`: must create a table with the exact same specifications as the one required for `ex00`, except that it must be named `ex02_movies`.

  It must return a page displaying "OK" if successful. Otherwise, it must display a message describing the problem.

- `127.0.0.1:8000/ex02/populate`: must insert the following data into the table created by the previous view:

  - episode_nb: 1 - title: The Phantom Menace - director: George Lucas - producer: Rick McCallum - release_date: 1999-05-19
  - episode_nb: 2 - title: Attack of the Clones - director: George Lucas - producer: Rick McCallum - release_date: 2002-05-16
  - episode_nb: 3 - title: Revenge of the Sith - director: George Lucas - producer: Rick McCallum - release_date: 2005-05-19
  - episode_nb: 4 - title: A New Hope - director: George Lucas - producer: Gary Kurtz, Rick McCallum - release_date: 1977-05-25
  - episode_nb: 5 - title: The Empire Strikes Back - director: Irvin Kershner - producer: Gary Kurtz, Rick McCallum - release_date: 1980-05-17

- episode_nb: 6 - title: Return of the Jedi - director: Richard Marquand - producer: Howard G. Kazanjian, George Lucas, Rick McCallum - release_date: 1983-05-25
- episode_nb: 7 - title: The Force Awakens - director: J. J. Abrams - producer: Kathleen Kennedy, J. J. Abrams, Bryan Burk - release_date: 2015-12-11

It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex02/display`: must display all data contained in the `ex02_movies` table in an `HTML` table, including any empty fields.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

Chapter VI

Exercise 03

	Exercise 03
Exercise 03: ORM - Data Insertion	
Turn-in directory : *ex*03/	
Files to turn in :	
Allowed functions :	

Create a new Django application named `ex03`. Inside it, create a Django model identical to the one created in `ex01`.

This application must include the following views, accessible via the following URLs:

- `127.0.0.1:8000/ex03/populate`: must insert the following data into this application's model:
    - episode_nb: 1 - title: The Phantom Menace - director: George Lucas - producer: Rick McCallum - release_date: 1999-05-19
    - episode_nb: 2 - title: Attack of the Clones - director: George Lucas - producer: Rick McCallum - release_date: 2002-05-16
    - episode_nb: 3 - title: Revenge of the Sith - director: George Lucas - producer: Rick McCallum - release_date: 2005-05-19
    - episode_nb: 4 - title: A New Hope - director: George Lucas - producer: Gary Kurtz, Rick McCallum - release_date: 1977-05-25
    - episode_nb: 5 - title: The Empire Strikes Back - director: Irvin Kershner - producer: Gary Kurtz, Rick McCallum - release_date: 1980-05-17
    - episode_nb: 6 - title: Return of the Jedi - director: Richard Marquand - producer: Howard G. Kazanjian, George Lucas, Rick McCallum - release_date: 1983-05-25

- episode_nb: 7 - title: The Force Awakens - director: J. J. Abrams - producer: Kathleen Kennedy, J. J. Abrams, Bryan Burk - release_date: 2015-12-11

It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex03/display`: must display all data contained in the `Movies` table in an `HTML` table, including any empty fields.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.



`During the evaluation, migrations will be applied before the tests are run.`

Chapter VII

Exercise 04

	Exercise 04
Exercise 04: SQL - Data Deletion	
Turn-in directory : $ex04/$	
Files to turn in :	
Allowed functions :	

Create an application named `ex04`. It must contain the following views, accessible via the following URLs:

- `127.0.0.1:8000/ex04/init`: must create a table with the exact same specifications as the one required for the `ex00` application, except that it must be named `ex04_movies`.

  It must return a page displaying "OK" if successful. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex04/populate`: must insert the data described in exercise `ex02` into the table created by the previous view.

  This view must reinsert any deleted data.

  It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex04/display`: must display all data contained in the `ex04_movies` table in an `HTML` table, including any empty fields.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

- `127.0.0.1:8000/ex04/remove`: must display an `HTML` form containing a drop-down list of movie titles and a `Submit` button labeled `Remove`.

The movie titles are those contained in the `ex04_movies` table.

When the form is submitted, the selected movie must be deleted from the database and the form must be redisplayed with the updated list containing the remaining movies.

If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

Chapter VIII

Exercise 05

	Exercise 05
Exercise 05: ORM - Data Deletion	
Turn-in directory : $ex05/$	
Files to turn in :	
Allowed functions :	

Create a new Django application named `ex05`. Create a model inside it identical to the one defined in `ex01`.

This application must include the following views, accessible via the following URLs:

- `127.0.0.1:8000/ex05/populate`: must insert the data described in exercise `ex03` into this application's model.

  This view must reinsert any deleted data.

  It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex05/display`: must display all data contained in the `Movies` table in an `HTML` table, including any empty fields.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

- `127.0.0.1:8000/ex05/remove`: must display an `HTML` form containing a drop-down list of movie titles and a `Submit` button labeled `Remove`.

  The movie titles are those contained in the `Movies` model of this application.

When the form is submitted, the selected movie is deleted from the database and the form is redisplayed with the updated list containing the remaining movies.

If there is no available data, or if an error occurs, the page must simply display `"No data available"`.



`During the evaluation, migrations will be applied before the tests are run.`

Chapter IX

Exercise 06

	Exercise 06
Exercise 06: SQL - Updating Data	
Turn-in directory : *ex*06/	
Files to turn in :	
Allowed functions :	

Create a new Django application named `ex06`. It must contain the following views, accessible via the following URLs:

- `127.0.0.1:8000/ex06/init`: must create a table with the exact same specifications as the one required for the `ex00` application, except that it must be named `ex06_movies` and must include the following additional fields:
  - `created`: a datetime field (date and time) that must automatically be set to the current date and time when the record is created.
  - `updated`: a datetime field (date and time) that must automatically be set to the current date and time when the record is created and automatically updated whenever the record is modified, thanks to the following trigger:

```
CREATE OR REPLACE FUNCTION update_changetimestamp_column()
RETURNS TRIGGER AS $$
BEGIN
NEW.updated = now();
NEW.created = OLD.created;
RETURN NEW;
END;
$$ language 'plpgsql';
CREATE TRIGGER update_films_changetimestamp BEFORE UPDATE
ON ex06_movies FOR EACH ROW EXECUTE PROCEDURE
update_changetimestamp_column();
```

- `127.0.0.1:8000/ex06/populate`: must populate the table created by the previous view with the data described in exercise `ex02`.

It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message describing the problem.

- `127.0.0.1:8000/ex06/display`: must display all data contained in the `ex06_movies` table in an `HTML` table.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

- `127.0.0.1:8000/ex06/update`: must handle both the display and submission of a form. This form must allow the user to select a movie from a drop-down list containing the movies stored in the `ex06_movies` table and to enter text into a second field. When the form is submitted, the view must replace the `opening_crawl` field of the selected movie with the text entered in the form in the `ex06_movies` table.

  If there is no available data, or if an error occurs, the page must simply display `"No data available"`.

Chapter X

Exercise 07

	Exercise 07
Exercise 07: ORM - Updating data	
Turn-in directory : *ex*07/	
Files to turn in :	
Allowed functions :	

Create a new Django app named `ex07`. Create a model identical to the one in `ex01` inside it, except that you will add the following fields:

- `created`: a datetime type (date and time) that, when created, must automatically be set to the current date and time.

- `updated`: a datetime type (date and time) that, when created, must automatically be set to the current date and time and automatically updated with each update.

This app must contain the following views accessible via these URLs:

- `127.0.0.1:8000/ex07/populate`: populates the model previously created with the same data as in `ex02`.

  It must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message stating the problem.

- `127.0.0.1:8000/ex07/display`: displays all the data included in the `Movies` table in an `HTML` table.

  If there is no available data, or an error, the page must simply display `"No data available"`.

- `127.0.0.1:8000/ex07/update`: must manage the submission and processing of a form. The latter must allow the user to choose a film from a drop-down menu containing the films included in the `Movies` table and to enter text in a second field.

When validating the form, the view must modify the `opening_crawl` field of the selected film with the text entered in the form and save it to the `Movies` model.

If there is no available data, or an error, the page must simply display `"No data available"`.



`During the evaluation, migrations will be applied before the tests.`

Chapter XI

Exercise 08

	Exercise 08
Exercise 08: SQL - Foreign Key	
Turn-in directory : *ex*08/	
Files to turn in :	
Allowed functions :	

Create a new Django app named `ex08`. This app must contain the following views accessible via these URLs:

- `127.0.0.1:8000/ex08/init`: must create two tables.
  The first must be named `ex08_planets` and include the following fields:
  - `id`: serial, primary key.
  - `name`: unique, variable character string, 64-byte maximum length, non-null.
  - `climate`: variable character string.
  - `diameter`: integer.
  - `orbital_period`: integer.
  - `population`: bigint.
  - `rotation_period`: integer.
  - `surface_water`: real.
  - `terrain`: variable character string, 128-byte maximum length.

  The second one must be named `ex08_people` and include the following fields:
  - `id`: serial, primary key.
  - `name`: unique, variable character string, 64-byte maximum length, non-null.

  - `birth_year`: variable character string, 32-byte maximum length.
  - `gender`: variable character string, 32-byte maximum length.
  - `eye_color`: variable character string, 32-byte maximum length.
  - `hair_color`: variable character string, 32-byte maximum length.
  - `height`: integer.
  - `mass`: real.
  - `homeworld`: variable character string, 64-byte maximum length, foreign key referencing the `name` column of the `ex08_planets` table.

- `127.0.0.1:8000/ex08/populate`: must populate both tables by copying the contents of the `people.csv` and `planets.csv` files into the corresponding tables, respectively: `ex08_people` and `ex08_planets`.

  This view must return a page displaying "OK" for each successful insertion. Otherwise, it must display an error message stating the problem.

- `127.0.0.1:8000/ex08/display`: displays all characters' names, their homeworld, as well as the climate when it is `windy` or moderately `windy`, sorted by character name in alphabetical order.

  If there is no available data, or an error, the page must simply display `"No data available"`.



`Read the documentation about the psycopg2 copy_from method.`

Chapter XII

Exercise 09

	Exercise 09
Exercise 09: ORM - Foreign Key	
Turn-in directory : *ex*09/	
Files to turn in :	
Allowed functions :	

Create a new Django app called `ex09` and create two models inside it. The first one must be named `Planets` and contain the following fields:

- `name`: unique, variable character string, 64-byte maximum length, non-null.
- `climate`: variable character string.
- `diameter`: integer.
- `orbital_period`: integer.
- `population`: bigint.
- `rotation_period`: integer.
- `surface_water`: real.
- `terrain`: text.

- `created`: a datetime field (date and time) that, when created, must automatically be set to the current date and time.

- `updated`: a datetime field (date and time) that, when created, must automatically be set to the current date and time and automatically updated whenever the record is modified.

This model must also redefine the `__str__()` method so that it returns the `name` attribute.

The second model you must create is named `People` and contains the following fields:

- `name`: character string, 64-byte maximum length, non-null.
- `birth_year`: character string, 32-byte maximum length.
- `gender`: character string, 32-byte maximum length.
- `eye_color`: character string, 32-byte maximum length.
- `hair_color`: character string, 32-byte maximum length.
- `height`: integer.
- `mass`: real.
- `homeworld`: foreign key referencing the `name` column of this app's `Planets` table.
- `created`: a datetime field (date and time) that, when created, must automatically be set to the current date and time.
- `updated`: a datetime field (date and time) that, when created, must automatically be set to the current date and time and automatically updated whenever the record is modified.

This model must also redefine the `__str__()` method so that it returns the `name` attribute.

In this app, you must also create a view accessible at the following address: `127.0.0.1:8000/ex09/d`

This view must display all characters' names, their homeworld, as well as the climate when it is `windy` or moderately `windy`, sorted alphabetically by character name, in an `HTML` table.

If there is no available data, the view must display the following text: `"No data available, please use the following command before using this page:"` followed by a command line.

This command line must be the one that has to be executed from the root of your repository in order to load all the data contained in the `ex09_initial_data.json` file (provided with today's resources) into the models previously created.

You must provide this file with your repository.



During the evaluation, migrations will be applied before the tests.

Chapter XIII

Exercise 10

	Exercise 10
Exercise 10: ORM - Many to Many	
Turn-in directory : $ex10/$	
Files to turn in :	
Allowed functions :	

Create a new Django app named `ex10` and create 3 models inside:

- `Planets` and `People`: both models must be identical to the ones in `ex09`.
- `Movies`: this model must be identical to the one in `ex01`, except you must add the field `characters`.
  This is a `many-to-many` relation with the `People` model. It allows listing every character in a film included in the `People` table.

The necessary fixtures to populate the models are included in the `ex10_initial_data.json` file provided with today's resources.

In this app, you will also create a view accessible via the following URL: `127.0.0.1:8000/ex10`. It must display a form featuring these mandatory fields:

- `Movies minimum release date`: date
- `Movies maximum release date`: date
- `Planet diameter greater than`: number
- `Character gender`: drop-down list showing the different values available in the `gender` field of the `People` model. The same value should not appear twice.

Once validated, the view must search, return and display the result(s).

A result is a character whose gender matches the `character gender` field, with a film they appear in, if the film was released between `Movies minimum release date` and `Movies maximum release date`, and their planet has a diameter greater than or equal to `Planet diameter greater than`.

If the search yields no result, the message `"Nothing corresponding to your research"` must appear. Each result must be displayed on a line with these elements (not necessarily in this order):

- Character's name
- Their gender
- Film title
- Name of homeworld
- Homeworld's diameter

For instance: the results for female characters whose films were released between "1900-01-01" and "2000-01-01" and whose homeworld has a diameter greater than 11000 are:

- A New Hope - Leia Organa - female - Alderaan - 12500
- The Phantom Menace - Padmé Amidala - female - Naboo - 12120
- Return of the Jedi - Leia Organa - female - Alderaan - 12500
- Return of the Jedi - Mon Mothma - female - Chandrila - 13500
- The Empire Strikes Back - Leia Organa - female - Alderaan - 12500



Several characters can be in the same film and one character can appear in several films. This is called a many-to-many relation. In this case, an intermediate table must be created between these tables. Each row of this intermediate table is a (unique) cross-reference: the first references a row in the films table, the second references a row in the characters table (or the other way around). Once your models are built and migrations applied, you will be able to see this table via the PostgreSQL console.

Chapter XIV

Submission and peer-evaluation

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double check the names of your folders and files to ensure they are correct.



`The evaluation process will happen on the computer of the evaluated group.`